НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ПОЛІТЕХНІКА»
МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ІНСТИТУТ КОМП'ЮТЕРНИХ СИСТЕМ
КАФЕДРА ІНФОРМАЦІЙНИХ СИСТЕМ

Протокол лабораторної роботи №1

«розгортання середовища та порівняльний аналіз мовних моделей»

з дисципліни: «Обчислювальні методи»

Виконав: студент групи АІ-242
Копитін І.Г
Перевірив: ас. Баранов О.М.

Одеса – 2026

Мета роботи: підготувати середовище для роботи з локальними й хмарними мовними моделями, вивчити відмінності між міркувальними та інструктивними моделями, зіставити їх за швидкістю та якістю відповідей, а також перевірити на практиці, як параметри temperature і top_p впливають на стабільність генерації.

Конфігурація середовища: лабораторну виконано під Windows 10. Локальні моделі запускались через Ollama 0.34.1 з використанням Python 3.12.10 та віртуального оточення venv. Було обрано дві локальні моделі – llama3.2:3b і qwen3:4b, а в ролі хмарної – gemini-3.6-flash.

Завдання 1. Ціна роздумів: міркувальна vs інструктивна модель

Щоб зіставити міркувальну та інструктивну моделі, до qwen3:4b і llama3.2:3b надсилались запити через ендпойнт /api/generate. Спершу обом моделям поставили просте фактичне запитання, а потім кожна з них по п’ять разів розв’язувала задачу на міркування. Перед вимірюваннями було зроблено один прогрівальний запит.

Результати відповіді на фактичне запитання подано в таблиці 1.

Таблиця 1 – Фактичне запитання «Столиця України? Відповідай одним словом.»

Показник	**qwen3:4b**	**llama3.2:3b**
response	Киев	Київ
eval_count	396	5
Час, с	81,72	0,90
Токенів/с	4,91	7,63

З таблиці 1 видно, що обидві моделі назвали столицю правильно, проте qwen3:4b для цього згенерувала набагато більше вихідних токенів і витратила суттєво більше часу.

Підсумки задачі на міркування (середнє за 5 запусків) наведено в таблиці 2.

Таблиця 2 – Задача на міркування (середнє з 5 запусків)

Показник	qwen3:4b	llama3.2:3b
Правильних із 5	5 / 5	1 / 5
eval_count	1230,4	151,0
Час, с	298,16	23,93

llama3.2:3b розв’язала задачу правильно лише в одному запуску з п’яти. У решті випадків вона видавала 50 грн, 100 грн, 100 грн, 5 грн та 105 грн, тобто з багатокроковою арифметикою модель справляється нестабільно. qwen3:4b, навпаки, відповіла правильно в усіх п’яти запусках, але ціною значно більшого часу та кількості токенів.

Відповіді на питання (крок 6):

1. Куди витрачаються додаткові токени міркувальної моделі?

Значну частину токенів qwen3:4b віддає на внутрішні міркування, що потрапляють у поле thinking, тоді як користувач бачить тільки підсумкову відповідь. Через це навіть на просте фактичне запитання вона генерує набагато більше токенів, ніж llama3.2:3b, що узгоджується з описом міркувальних моделей у методичних вказівках.

2. На яких задачах міркувальні моделі мають перевагу?

Вони виграють там, де треба пройти кілька логічних або математичних кроків, розібратися з умовою й перевірити результат. Для простих фактичних запитів їх застосовувати недоцільно, бо вони марнують час і токени.

3. Чим навчання міркувальної моделі відрізняється від інструктивної?

Інструктивну модель вчать насамперед чітко виконувати команди користувача та швидко видавати готову відповідь. Міркувальну додатково налаштовують на побудову проміжного ланцюжка розв’язання, що допомагає зі складнішими задачами.

4. Чи означає більша кількість токенів, що модель розумніша?

Ні. Велика кількість токенів свідчить лише про обсяг згенерованого тексту. Якість відповіді слід оцінювати за правильністю, швидкістю та відповідністю завданню.

5. Чим пояснити різницю у швидкості генерації?

На швидкість впливають архітектура та розмір моделі, спосіб генерації й можливості комп'ютера. У нашому досліді qwen3:4b ще й генерувала внутрішні міркування, тому її загальний час відповіді був значно довшим.

Висновок за завданням 1

Міркувальні моделі варто залучати до задач, де потрібні логічні обчислення та кілька кроків розв'язання. У моєму досліді qwen3:4b розв'язала задачу 5 разів із 5, а llama3.2:3b – лише 1 раз із 5, зате працювала набагато швидше. Отже, прості запити краще віддавати інструктивній моделі, а складніші – міркувальній.

Завдання 2. Роль system-повідомлення: керування поведінкою моделі

Щоб оцінити вплив system-повідомлення, один і той самий запит варіанта 10 було надіслано моделям llama3.2:3b та qwen3:4b з двома різними системними інструкціями: спочатку «Ти лаконічний технічний асистент», потім «Відповідай лише українською, максимум двома реченнями».

Вплив system-повідомлення на вигляд відповіді показано в таблиці 3.

Таблиця 3 – Вплив system-повідомлення

Модель	System	Довжина, символів	Мова відповіді	Кількість речень	Інструкцію виконано?
llama3.2:3b	лаконічний асистент	≈1514	переважно українська, з домішками	≈11	частково
llama3.2:3b	українською, 2 речення	313	українська, з помилками	2	так, за форматом
qwen3:4b	лаконічний асистент	283	українська	2	так

Модель	System	Довжина, символів	Мова відповіді	Кількість речень	Інструкцію виконано?
qwen3:4b	українською, 2 речення	168	українська	1	так

Таблиця 3 показує, що чітка вимога щодо мови та кількості речень виконувалась краще, ніж розмите прохання бути лаконічним асистентом. Витрати токенів для кожної моделі та кожного system-повідомлення подано в таблиці 4.

Таблиця 4 – Витрати токенів

Модель	System	prompt_tokens	completion_tokens	total_tokens
llama3.2:3b	лаконічний асистент	59	457	516
llama3.2:3b	українською, 2 речення	63	92	155
qwen3:4b	лаконічний асистент	65	1688	1753
qwen3:4b	українською, 2 речення	74	845	919

Як видно з таблиці 4, після уточнення обмеження довжини completion_tokens зменшилась в обох моделей. При цьому qwen3:4b усе одно генерувала помітно більше вихідних токенів, ніж llama3.2:3b.

Відповіді на питання (крок 6)

1. Яка модель точніше дотрималась обмеження «максимум двома реченнями»?

Обмеження виконали обидві моделі: llama3.2:3b написала два речення, а qwen3:4b – одне, тож обидві відповіді вкладаються в задану вимогу.

2. Чи траплялось, що модель відповідала не українською всупереч прямій вказівці?

За прямої вказівки обидві моделі здебільшого трималися української. У відповіді Llama були граматичні помилки, а Qwen вжила технічний термін callback-функцій, проте основною мовою лишалась українська.

3. Як змінився prompt_tokens після зміни system-повідомлення і чому?

У Llama значення зросло з 59 до 63, у Qwen – з 65 до 74. Причина в тому, що друге system-повідомлення довше та містить більше вимог.

4. Чому в qwen3:4b completion_tokens набагато більше, ніж у llama3.2:3b, хоча видима відповідь має схожу довжину?

qwen3:4b – міркувальна модель, тому частину вихідних токенів вона витрачає на внутрішні міркування, які користувач не бачить повністю. Через це completion_tokens може бути великим навіть за короткої підсумкової відповіді.

Відповіді на питання (крок 7)

1. Чому моделі по-різному виконують прямі інструкції щодо довжини та мови відповіді?

Моделі різняться архітектурою, способом навчання та вмінням слідувати інструкціям. У моєму досліді конкретна вимога «лише українською, максимум двома реченнями» спрацювала краще, ніж загальне прохання бути лаконічним.

2. Чим system-повідомлення відрізняється від user-повідомлення для моделі і чи має воно вищий пріоритет?

System-повідомлення задає загальні правила поведінки – мову, стиль, формат та інші обмеження, а user-повідомлення містить саме завдання користувача. Зазвичай пріоритет system вищий, тому модель має виконувати запит користувача в межах цих правил.

3. Чому міркувальна модель витрачає більше вихідних токенів, навіть якщо видима відповідь така сама за довжиною?

Перед формуванням підсумкової відповіді qwen3:4b проводить внутрішні міркування, на які йдуть токени. Тому в моєму досліді коротка відповідь Qwen мала 845–1688 completion tokens, а в Llama – лише 92–457.

4. Що буде, якщо дати моделі суперечливі інструкції в system і user?

Загалом модель має триматися system-інструкції, а суперечливу частину user-запиту ігнорувати. Однак малі моделі можуть дотримуватись такої ієрархії неідеально й подекуди частково виконувати інструкцію з нижчим пріоритетом.

5. Чому малі моделі гірше за великі дотримуються вимог щодо мови відповіді?

У малих моделей менше параметрів, і вони слабше утримують одночасно зміст запиту та додаткові обмеження на мову чи формат. Через це вони частіше змішують мови або вигадують дивні формулювання, що ми й бачили у відповідях llama3.2:3b.

Висновок за завданням 2

System-повідомлення помітно впливає на стиль, мову та довжину відповіді, але повного виконання інструкції не гарантує. У моєму досліді конкретне обмеження «лише українською, максимум двома реченнями» працювало краще за загальну вимогу бути лаконічним, тому для стабільного формату інструкції слід формулювати якомога конкретніше.

Завдання 3. Локальна vs хмарна модель: єдиний інтерфейс

Локальну й хмарну моделі порівнювали на однаковому запиті варіанта 10: llama3.2:3b і gemini-3.6-flash викликались через єдиний OpenAI-сумісний інтерфейс. Усі відмінності між провайдерами винесено у файл providers.py, тому основний код виклику моделі залишався незмінним.

Результати порівняння наведено в таблиці 5.

Таблиця 5 – Порівняння локальної та хмарної моделі

Показник	Локальна	Хмарна
Назва моделі	llama3.2:3b	gemini-3.6-flash
Час відповіді, с	57,37	9,51
completion_tokens	200	291

Показник	Локальна	Хмарна
Довжина відповіді, символів	664	868
Мова відповіді	переважно українська, є мовні помилки та сторонній символ 順	українська
Фактичних помилок у відповіді	є	суттєвих не виявлено

За таблицею 5 хмарна модель відповіла набагато швидше й дала природнішу українську відповідь, тоді як локальна припустилась і мовних, і змістових помилок.

Відповіді на питання (крок 5)

1. Скільки рядків коду довелося змінити під час переходу з локальної моделі на хмарну?

В основному коді виклику змінювати нічого не потрібно: функція create() однаково працює для обох провайдерів. Різниця зосереджена у словнику PROVIDERS, де для хмарної моделі задано інші base_url, API-ключ і назву моделі.

2. Яка відповідь повніша й точніша? Наведіть приклад.

Повнішою й точнішою була відповідь gemini-3.6-flash: вона правильно описала синхронне виконання як послідовне з очікуванням завершення попередньої операції, а асинхронне – як неблокуюче. llama3.2:3b помилково пов’язала асинхронність із виконанням в окремих потоках і сформулювала думку плутано.

3. Чи були в локальної моделі фактичні помилки або проблеми з українською?

Так. У локальній відповіді траплялись і змістові неточності, і мовні проблеми: неприродні звороти, помилки в словах і навіть сторонній символ 順

4. За рахунок чого хмарна модель швидша, якщо запит іде через інтернет?

Хмарна модель працює на значно потужнішому серверному обладнанні, оптимізованому під нейромережі, тому обчислення займають набагато менше часу. У нашому випадку навіть з урахуванням затримки мережі Gemini відповіла за 9,51 с, а локальна Llama – за 57,37 с.

Відповіді на питання (крок 6)

1. Чому формат запитів OpenAI став стандартом, який підтримують інші провайдери, зокрема Ollama?

Формат OpenAI API поширився завдяки простій і зрозумілій структурі запитів, яку почали підтримувати й інші сервіси. Тому той самий клієнтський код можна використовувати з різними моделями, змінюючи лише адресу сервера, ключ і назву моделі.

2. Наскільки розмір моделі (3 млрд проти 70 млрд параметрів) впливає на якість порівняно з іншими чинниками?

Більша кількість параметрів зазвичай дає моделі ширші можливості для складних запитів, але розмір – не єдиний чинник якості. Не менш важливі метод навчання, якість даних, архітектура та налаштування генерації.

3. Чим пояснюється, що хмарна модель швидша попри мережеву затримку?

Хмарні моделі виконуються на потужному серверному обладнанні, яке спеціально підготовлене для нейромереж. Тому навіть з урахуванням передавання запиту через інтернет загальний час може бути меншим, ніж обчислення на звичайному домашньому комп’ютері.

4. Які ризики для приватності виникають при використанні хмарних API і що з цим роблять у комерційних проєктах?

Під час роботи з хмарним API текст запиту потрапляє до стороннього провайдера, тому конфіденційні дані не можна надсилати безконтрольно. У комерційних проєктах застосовують мінімізацію й анонімізацію даних,

контроль доступу та перевіряють політику зберігання інформації в провайдера.

5. Що таке rate limit у безкоштовних тарифах і як його обходять?

Rate limit – це обмеження кількості запитів або токенів за певний проміжок часу. Розробники скорочують кількість зайвих запитів, застосовують кешування, повторні спроби із затримкою або розподіляють навантаження між кількома сервісами.

Висновок за завданням 3

Локальну модель доцільно обирати, коли важливі приватність даних, робота без інтернету або відсутність витрат на API. Хмарну доцільніше брати, коли потрібні вища якість, краща українська мова та швидше виконання складних запитів, за умови що передавання даних стороннім сервісам припустиме.

Завдання 4. Вимірювання швидкості: TTFT і залежність від довжини запиту

Для локальної llama3.2:3b та хмарної gemini-3.6-flash вимірювали час до першого токена (TTFT), загальний час відповіді та швидкість генерації в символах за секунду. Для кожного типу промпту робили по три повтори й брали медіану, а перед серією виконували прогрівальний запит.

Результати вимірювань наведено в таблиці 6.

Таблиця 6 – Результати вимірювання швидкості

Розміщення	Модель	Промпт	TTFT, с	Всього, с	Симв./с
локальна	llama3.2:3b	короткий	3,564	11,857	11,8
локальна	llama3.2:3b	середній	3,488	20,601	15,8
локальна	llama3.2:3b	довгий	3,529	156,600	15,7
хмарна	gemini-3.6-flash	короткий	1,847	1,984	19,6
хмарна	gemini-3.6-flash	середній	5,776	6,552	82,6
хмарна	gemini-3.6-flash	довгий	7,841	14,749	241,9

Таблиця 6 показує, що перевага залежить від обраного показника. На короткому промпті TTFT хмарної моделі був меншим, але на середньому й довгому першим токен швидше з’являвся в локальної. Водночас за швидкістю подальшої генерації, особливо на довгому промпті, gemini-3.6-flash суттєво випередила локальну модель.

Окремо для локальної моделі порівняли холодний і теплий старт (таблиця 7).

Таблиця 7 – Холодний і теплий старт локальної моделі

Показник	**Холодний старт**	**Теплий старт**
TTFT, с	18,928	4,192
Всього, с	20,851	14,836

Після прогріву TTFT локальної моделі впав з 18,928 до 4,192 с, що вказує на значний вплив завантаження моделі в пам’ять під час першого запуску.

Швидкість генерації для промптів різної довжини також показано на рисунку 1.

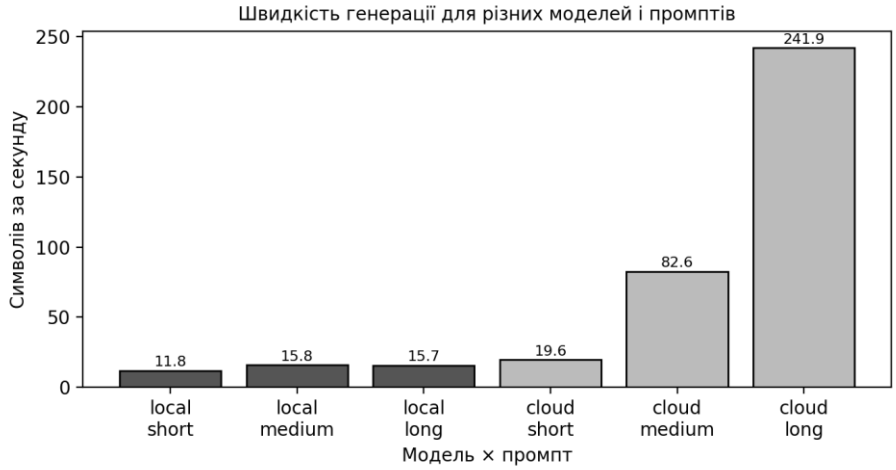


Рисунок 1 – Швидкість генерації локальної та хмарної моделей для промптів різної довжини

Відповіді на питання (крок 6)

1. У якої моделі кращий TTFT і чому?

Однозначного лідера немає. На короткому промпті кращий TTFT показала gemini-3.6-flash (1,847 с проти 3,564 с), а на середньому та довгому менший TTFT мала llama3.2:3b: 3,488 і 3,529 с проти 5,776 і 7,841 с відповідно.

2. Чи збігається «найшвидша за TTFT» з «найшвидшою за символами/с»? Якщо ні – поясніть.

Ні. На середньому й довгому промптах менший TTFT мала локальна модель, але хмарна генерувала текст набагато швидше. Наприклад, на довгому промпті Gemini видала 241,9 симв./с, а Llama – 15,7 симв./с.

3. Як змінилось співвідношення локальної та хмарної моделі від короткого промпту до довгого?

На короткому промпті різниця в загальному часі була помітною, а на довгому – набагато більшою: локальна модель витратила 156,6 с, хмарна – лише 14,749 с.

4. Наскільки відрізняються три повтори одного виміру? Чи вистачило б одного запуску?

Окремі запуски можуть різнитися через кешування, стан моделі, навантаження на систему й мережу, тому одного виміру недостатньо. Три повтори з медіаною дають стабільніший результат і зменшують вплив випадкових затримок.

5. Чому холодний старт повільніший за теплий?

Під час холодного старту модель спершу завантажується в оперативну пам’ять і готується до роботи, тому TTFT більший. У нашому тесті він зменшився з 18,928 до 4,192 с після прогріву.

Відповіді на питання (крок 7)

1. Що фізично відбувається під час TTFT і чому його вимірюють окремо від загальної швидкості генерації?

Протягом TTFT система приймає запит, обробляє вхідні токени, виконує початкові обчислення та готує перший вихідний токен. Його вимірюють окремо, бо він показує, скільки користувач чекає до появи першого тексту, а не те, як швидко йде подальша генерація.

2. Чому хмарна модель може мати гірший TTFT, але кращу швидкість генерації?

Хмарний запит має додаткову мережеву затримку, через що перший токен іноді з’являється пізніше. Але коли генерація розпочалась, потужне серверне обладнання зазвичай видає токени значно швидше за звичайний локальний комп’ютер.

3. Чому перший запуск локальної моделі повільніший і що таке холодний старт?

Холодний старт – це запуск, коли модель ще не завантажена в пам’ять і системі потрібно її підготувати. Наступні запити швидші, бо модель уже розміщена в пам’яті й готова генерувати.

4. Чому під час вимірювання продуктивності беруть медіану кількох запусків?

Окремий запуск може випадково виявитись повільнішим чи швидшим через фонові процеси, кеш, мережу або навантаження на сервер. Медіана кількох вимірів менше залежить від таких випадкових відхилень.

5. Чому обробка вхідного запиту швидша за генерацію відповіді такої самої довжини?

Вхідний текст модель може опрацьовувати великими блоками паралельно, а вихідні токени генеруються послідовно, один за одним. Тому створення довгої відповіді зазвичай триває довше, ніж аналіз вхідного тексту подібного обсягу.

Висновок за завданням 4

Для інтерактивного чат-бота важливіший TTFT, адже користувач одразу відчуває затримку перед появою відповіді. Для фонової обробки великих обсягів тексту важливіша швидкість генерації, бо саме вона визначає загальний час виконання довгих завдань.

Завдання 5. Оцінювання якості моделей за рубрикою

Для порівняння якості моделей використали шість типів завдань: перевірка фактичних знань, дотримання формату JSON, стиснення наданого тексту, генерація коду, багатокрокове логічне міркування та якість української мови. Кожне завдання виконувалось двічі для llama3.2:3b, qwen3:4b і хмарної gemini-3.6-flash. У завданні на фактичні знання правильна відповідь – 2007 рік: за даними Apple Newsroom, перший iPhone надійшов у продаж у США 29 червня 2007 року [1]. JSON-відповіді із завдання на дотримання формату додатково перевірялись функцією json.loads(), причому окремо для обох прогонів кожної моделі.

Оцінки за рубрикою наведено в таблиці 8.

Таблиця 8 – Оцінка якості за рубрикою

Завдання	llama3.2:3b	qwen3:4b	Хмарна модель
1. Факт	2	2	2
2. Формат	2	2	2
3. Стиснення	0	2	2
4. Код	2	0	1
5. Логіка	0	2	2
6. Українська	0	0	1
Разом /12	6/12	8/12	10/12

Найвищу суму балів у цьому експерименті отримала gemini-3.6-flash – 10 із 12. qwen3:4b набрала 8 балів, а llama3.2:3b – 6. Щоб оцінити стабільність, порівняли бали двох незалежних прогонів (таблиця 9).

Таблиця 9 – Розбіжність оцінок між двома прогонами (так / ні)

Завдання	llama3.2:3b	qwen3:4b	Хмарна модель
1. Факт	ні	ні	ні
2. Формат	ні	ні	ні
3. Стиснення	ні	ні	ні
4. Код	ні	ні	ні
5. Логіка	ні	ні	ні
6. Українська	ні	ні	ні

Таблиця 9 свідчить, що в межах двох прогонів оцінки за рубрикою не відрізнялись у жодної з моделей, хоча самі тексти відповідей подекуди були різними.

Відповіді на питання (крок 6)

1. Яка модель набрала найбільше балів і де виникла основна різниця?

Найбільше балів – 10 із 12 – набрала gemini-3.6-flash. Основна різниця між моделями проявилась у завданнях на стиснення тексту, генерацію коду, багатокрокову логіку та якість української мови.

2. На яких завданнях оцінки двох прогонів розійшлися? Що це говорить про надійність моделі?

В нашому експерименті розбіжностей не було в жодному завданні для жодної моделі. Отже, за цією рубрикою результати виявились стабільними, хоча формулювання відповідей могли відрізнятись.

3. Чи виконала хоч одна модель вимогу «без markdown» у завданні 2?

Так, вимогу виконали всі три моделі: їхні JSON-відповіді без жодних правок оброблялись через json.loads().

4. Чи були проблеми з українською – русизми, кальки, перехід на іншу мову?

Так. У llama3.2:3b траплялись переходи на інші мови, сторонні символи та кострубаті конструкції, а qwen3:4b подекуди змішувала українську з англійською й вживала неприродні звороти. gemini-3.6-flash писала значно природніше, хоча мала окремі неточності в термінології.

Крок 7. Приклад провалу

Повний текст запиту: сервер обробляє 1200 запитів/хв, 15% дають помилку, з них 40% повторюються успішно. Скільки запитів втрачено за годину? Покажи хід розв'язання.

Повний текст відповіді моделі llama3.2:3b:

Дякуюм за запит!
Зрозумію, що нам потрібно знайдити число запитів, які втрачаються за годину.
Початківий, ми маємо наступність істот:
1. Обробка запитів на хвилину: 1200 запитів/хв.
2. Суспільний рівень помилок: 15%.
3. Суспільний рівень повторювання успішно: 40%.
Вихід з цієї проблеми можна здійснити за допомогою наступного ходу:
*1. Кількість помильних запитів на хвилину: 15% * 1200 запитів/хв = 0,15 * 1200 = 180 запитів/хв.*
*2. Кількість повторно успішно оброблених запитів на хвилину: 40% * 1200 запитів/хв = 0,4 * 1200 = 480 запитів/хв.*
3. Кількість втраченних запитів на хвилину: 180 запитів/хв - 480 запитів/хв = -300 запитів/хв.
*4. Кількість втраченних запитів на годину: -300 запитів/хв * 3600 хв в годі = -1080000 запитів в годину.*
Теоретически, за годину втрачається 1080000 запитів.

Правильна відповідь: 6480 запитів за годину.

Правильний хід: 1200 × 15% = 180 помилкових запитів за хвилину. З них 40% повторюються успішно, тож втрачаються 60%: 180 × 0,60 = 108 запитів за хвилину. За годину: 108 × 60 = 6480.

Модель заплуталась у багатокроковій арифметиці й застосувала 40% до всіх 1200 запитів замість того, щоб брати їх лише від 180 помилкових. Далі

помилка накопичувалась, і в результаті вийшла від'ємна кількість втрачених запитів та хибна кінцева відповідь. Це демонструє, що правдоподібно оформлений хід розв'язання ще не гарантує правильності обчислень.

Відповіді на питання (крок 8)

1. Які ще підходи до оцінювання якості мовних моделей застосовують на практиці, крім ручної рубрики?

Крім ручної рубрики, використовують автоматичні тести, набори еталонних відповідей, спеціалізовані бенчмарки та порівняння відповідей кількох моделей. Також застосовують оцінювання людьми або іншою моделлю за заздалегідь визначеними критеріями.

2. Чому моделі часто ігнорують пряму інструкцію не використовувати markdown у JSON?

У навчальних даних код і JSON дуже часто оформлені markdown-блоками. Через це звичний шаблон відповіді іноді виявляється сильнішим за окрему інструкцію користувача.

3. Чому малі моделі гірше пишуть українською, ніж англійською?

В навчальних даних англомовних текстів зазвичай набагато більше, ніж україномовних, а мала модель гірше узагальнює рідкісні мовні конструкції. Тому частіше з'являються русизми, кальки, неприродні фрази та змішування мов.

4. Чому одна й та сама модель на однаковий запит може давати відповіді різної якості?

У генерації є випадковий компонент, особливо коли temperature більша за нуль. Тому на той самий запит модель може обрати інші наступні токени й побудувати інший хід відповіді.

5. Що таке галюцинація і чому модель радше вигадає факт, ніж визнає незнання?

Галюцинація – це правдоподібна, але хибна або вигадана відповідь. Модель генерує найімовірніше продовження тексту й не має вбудованого

механізму перевірки істинності кожного факту, тому іноді впевнено видає те, чого насправді не знає.

Висновок за завданням 5

Оцінювання за рубрикою виявилось достатньо надійним, бо дозволило порівнювати моделі за однаковими критеріями, а не за суб'єктивним враженням. Найпоказовішими були завдання на логіку, формат JSON та українську мову, де відмінності між моделями проявились найкраще. Для роботи з україномовним текстом доцільно обирати gemini-3.6-flash, оскільки вона показала найкращу мову та найвищий результат – 10 із 12 балів.

Завдання 6. Керування випадковістю: temperature і top_p

Експерименти з параметрами temperature і top_p проводились на локальній моделі llama3.2:3b. Творчим завданням було придумати назву застосунку для пішохідних екскурсій, фактичним – запитання про рік відкриття будівлі Одеського оперного театру. Правильна відповідь для фактичного завдання – 1887 рік: за офіційною історією Одеського національного академічного театру опери та балету, будівництво нової будівлі завершилось у вересні 1887 року, а 1 жовтня 1887 року глядачі вперше заповнили залу [2].

Вплив параметрів на різноманітність відповідей наведено в таблиці 10.

Таблиця 10 – Вплив параметрів на різноманітність відповідей

Параметр	**Значення**	**Унікальних із 5 (творче)**	**Правильних із 5 (фактичне)**
temperature	0.0	1	0
temperature	0.7	5	-
temperature	1.5	5	0
top_p	0.1	1	-
top_p	0.5	5	-
top_p	1.0	5	-

З таблиці 10 видно, що за temperature = 0,0 усі п’ять творчих відповідей збіглись, а за 0,7 і 1,5 було отримано по п’ять різних. Для top_p картина схожа: при 0,1 усі відповіді однакові, а при 0,5 і 1,0 – усі п’ять різні. На фактичному завданні при temperature = 0,0 модель п’ять разів повторила однакову, але хибну відповідь 1856. При temperature = 1,5 з’явились різні роки – 1876, 1849, 1856, 1907 та 1846, але правильного 1887 не було жодного разу. Отже, низька температура підвищила стабільність відповіді, проте не забезпечила її фактичної правильності.

Для окремої перевірки відтворюваності фактичний запит виконали десять разів при temperature = 0,0 (таблиця 11).

Таблиця 11 – Перевірка відтворюваності

Показник	**Значення**
Кількість запусків	10
temperature	0.0
Унікальних відповідей	1
Усі відповіді однакові (так / ні)	так

В усіх десяти запусках модель видала ту саму відповідь – 1856. Тобто результат був повністю відтворюваним, але фактично хибним.

Відповіді на питання (крок 7)

1. Яку температуру поставити агенту, що має повертати JSON, і чому?

Доцільно встановити низьку температуру, наприклад 0.0 або близьку до нуля. Для JSON важливі передбачуваність і стабільність формату, а висока температура збільшує випадковість.

2. А генератору варіантів заголовків?

Для генератора заголовків краще взяти вищу температуру, наприклад 0.7–1.0. У моєму досліді при 0.7 і 1.5 було по 5 унікальних відповідей із 5, а при 0.0 модель повторювала одну й ту саму.

3. Чим вплив top_p відрізняється від впливу temperature за характером, а не за силою?

temperature змінює сам розподіл імовірностей, роблячи вибір більш або менш випадковим. top_p не змінює співвідношення ймовірностей, а лише обмежує набір токенів, з яких можна вибирати наступний.

4. Чи означає temperature=0 повну відтворюваність? Що показав експеримент із 10 запусків?

Математичної гарантії це не дає, але в моєму експерименті всі 10 запусків при temperature=0.0 дали ту саму відповідь 1856. Тож у цій серії результат був повністю відтворюваним.

5. Наведіть найбезглуздішу відповідь при temperature=1.5 і поясніть, чому вона з'явилась.

Однією з найбезглуздіших була назва «Теребре», яка не має зрозумілого змісту для застосунку пішохідних екскурсій. Вона могла з'явитись, бо висока температура підвищує ймовірність вибору малоймовірних токенів і незвичних поєднань слів.

Відповіді на питання (крок 8)

1. Що математично робить temperature з розподілом імовірностей наступного токена?

temperature змінює «гостроту» розподілу перед вибором наступного токена. Низьке значення робить найімовірніші токени ще домінантнішими, а високе вирівнює розподіл і збільшує шанс вибору менш імовірних варіантів.

2. Чим top_p відрізняється від temperature за механізмом?

temperature змінює відносні ймовірності токенів, а top_p відсікає малоймовірний «хвіст» і залишає лише найімовірніші токени, сумарна ймовірність яких досягає заданого порога. Механізми різні, хоча обидва впливають на різноманітність.

3. Чи варто змінювати обидва параметри одночасно, чи краще налаштовувати один?

На практиці можна змінювати обидва, але для зрозумілого налаштування краще спочатку варіювати один параметр, залишаючи інший фіксованим. Так простіше визначити, що саме змінило поведінку моделі.

4. Чому навіть при temperature=0 модель не гарантує абсолютно однакову відповідь?

На результат можуть впливати особливості обчислень із плаваючою комою, паралелізація та реалізація самого inference. У моєму досліді всі 10 запусків збіглись, але універсальною гарантією це не є.

5. Які значення temperature використовують для структурованих даних, а які для творчого тексту?

Для структурованих даних беруть низькі значення, близькі до 0.0, щоб зменшити випадковість і стабілізувати формат. Для творчих задач частіше обирають середні або високі значення, наприклад 0.7–1.0, щоб отримувати більше різних варіантів.

Висновок за завданням 6

У наступних лабораторних, де модель повертатиме структуровані дані, варто встановлювати низьку temperature, близьку до 0.0, та помірний top_p, щоб зменшити випадковість і підвищити стабільність формату. Експеримент показав, що при temperature=0.0 відповідь була однаковою у всіх 10 запусках, а при temperature=1.5 відповіді ставали різноманітнішими й подекуди беззмістовними. Типові налаштування можуть не підходити, бо не враховують конкретну задачу: для JSON потрібна передбачуваність, а для творчої генерації – різноманітність.

Підсумкове порівняння моделей

Узагальнені результати всіх експериментів зведено в таблиці 12.

Таблиця 12 – Підсумкове порівняння моделей

Критерій	**Локальна мала (3B)**	**Локальна більша (4B/8B)**	**Хмарна**
Швидкість генерації	У benchmark: приблизно 11,8–15,8 симв./с	Повний benchmark окремо не проводився; у раньому фактичному тесті ≈ 4,91 ток./с, модель працювала значно повільніше	У benchmark: приблизно 19,6–241,9 симв./с, найвища швидкість на довгому запиті
TTFT	Приблизно 3,49–3,56 с у benchmark	Окремо не вимірювався	1,85 с для короткого, 5,78 с для середнього, 7,84 с для довгого промпту
Якість, балів /12	6/12	8/12	10/12
Стабільність формату (завдання 2)	Висока: обидва JSON парсились через json.loads() без правок	Висока: обидва JSON парсились без правок	Висока: обидва JSON парсились без правок
Потрібні ресурси	Працює на локальному ПК; потребує оперативної пам’яті та обчислювальних ресурсів	Потребує більше локальних ресурсів і в моїх тестах працювала значно повільніше	Основні обчислення виконуються на сервері провайдера; локальні вимоги мінімальні
Вартість	Немає оплати за кожен API-запит, але використовуються ресурси та електроенергія власного ПК	Так само: немає оплати за локальний запит, але потрібні більші ресурси ПК	Залежить від тарифу хмарного API; може існувати оплата за використання
Куди йдуть ваші дані	Залишаються на локальному комп’ютері	Залишаються на локальному комп’ютері	Запит передається зовнішньому хмарному провайдеру
Працює без інтернету	Так	Так	Ні

Критерій	Локальна мала (3B)	Локальна більша (4B/8B)	Хмарна
Коли обирати саме її	Коли важлива приватність, автономність, робота без інтернету й задача не потребує максимальної якості	Коли потрібна локальна модель з кращим міркуванням, ніж у малої, і допустимі більші витрати часу та ресурсів	Коли пріоритетом є якість, швидка генерація великих відповідей і краща українська, а доступ до інтернету та передача запитів стороннім сервісам допустимі

Загальні висновки

1) Локальну й хмарну моделі можна використовувати через єдиний OpenAI-сумісний програмний інтерфейс, тому перехід між провайдерами не потребує змін в основній логіці програми. Це дає змогу будувати системи з перемиканням між локальним і хмарним виконанням.

2) Продуктивність мовної моделі треба оцінювати окремо за TTFT і за швидкістю подальшої генерації. В експерименті llama3.2:3b мала менший TTFT на середньому та довгому промптах, але gemini-3.6-flash значно випереджала її за швидкістю генерації довгих відповідей. Тож одного показника недостатньо, щоб назвати модель «швидшою».

3) Міркувальний режим доцільний передусім для задач із багатокроковими логічними або математичними обчисленнями. На простому фактичному запиті qwen3:4b витратила значно більше токенів і часу, зате в задачі на міркування відповіла правильно в усіх п’яти запусках.

4) Текстова інструкція не гарантує точного дотримання заданого формату. У завданні на генерацію коду qwen3:4b додавала значний обсяг зайвого тексту, а gemini-3.6-flash використовувала markdown-оформлення. Тому, коли відповідь призначена для програми, формат потрібно перевіряти автоматично.

5) Повторне виконання однакового запиту може давати різні результати, тому одного запуску недостатньо для надійного вимірювання. Для

продуктивності використовувались кілька повторів із медіаною, а для якості кожне завдання виконувалось двічі.

6) Низьке значення temperature підвищує стабільність відповіді, але не гарантує її правильності. При temperature = 0,0 модель десять разів поспіль дала однакову хибну відповідь 1856, а при temperature = 1,5 відповіді стали різноманітнішими, проте теж лишились неправильними.

7) Оцінювання за рубрикою показало різні профілі моделей: llama3.2:3b отримала 6/12, qwen3:4b – 8/12, gemini-3.6-flash – 10/12. Найпомітніше відрізнялись завдання на багатокрокову логіку, генерацію коду, роботу з наданим текстом і якість української мови.

8) Вибір між локальною та хмарною моделлю залежить від вимог конкретної задачі. Локальну доцільно брати, коли пріоритетні приватність, автономність і робота без інтернету. Хмарну варто обирати, коли важливіші якість, швидкість генерації та краща робота зі складними або україномовними запитами, а передача даних зовнішньому провайдеру допустима.

Список використаних джерел

1. Apple Newsroom. iPhone Premieres This Friday Night at Apple Retail Stores. June 28, 2007. https://www.apple.com/newsroom/2007/06/28iPhone-Premieres-ThisFriday-Night-at-Apple-Retail-Stores/

2. Одеський національний академічний театр опери та балету. Історія театру. https://operahouse.od.ua/about/history/